CHARUSAT®

CHAROTAR UNIVERSITY OF SCIENCE AND TECHNOLOGY

अमृतं तु विद्या

FACULTY OF TECHNOLOGY AND ENGINEERING

DEVANG PATEL INSTITUTE OF ADVANCE TECHNOLOGY AND RESEARCH

DEPARTMENT OF COMPUTER ENGINEERING

A.Y 2026 – 27 Semester – 3

LAB MANUAL

CSUC201 Fundamentals of Data Structures and Algorithms

DEPSTAR

Practical 2: Linear and Binary Search: Iterative and Recursive Approaches

Practical Aim

To implement linear and binary search using both iterative and recursive approaches.

Problem 1

A security guard at a parking lot checks vehicles one by one from the entrance to find a car with specific license plate. Sometimes he starts from the entrance, sometimes he calls a helper who starts from where the guard left off. Given a list of license plates and a target plate, implement both approaches — one that checks plates one by one from the start, and one where the function calls itself to continue checking — and report the position of the target plate if found. Describe the approach you used. What happens in your solution if the target plate appears more than once in the list? Does it find the first occurrence, the last, or just any one?

Problem 2

A librarian maintains a sorted catalog of book codes and needs to locate a specific code quickly. Instead of going through every book, she opens the catalog to the middle, checks whether the target is to the left or right, and repeats. Given a sorted list of book codes and a target code, implement both approaches — one using a loop and one where the function calls itself — and report the position of the target code. Describe the approach you used. What is the one condition your input must satisfy for this approach to work correctly? What would go wrong if that condition was not met?

For Code:

https://github.com/Brijesh2212/FDSA_A_SHP/tree/main/PRAC-2

Output:

Problem 1

```
PS D:\dsa practicals> cd "d:\dsa practicals\" ; if ($?) { g++ prac2-1.cpp -o prac2-1 } ; if ($?) { .\prac2-1 }
Enter the number of cars in the parking lot
3
Enter numberplates of cars
1111
2222
3333
Enter the number of car you want to find
2222
The Car with the numberplate 2222 is at the 2 position
```

Problem 2

```
PS D:\dsa practicals> cd "d:\dsa practicals\" ; if ($?) { g++ prac2-2.cpp -o prac2-2 } ; if ($?) { .\prac2-2 }
Enter number of book codes: 5
Enter book codes in sorted order:
10
12
14
16
18
Enter book code to search: 14
Iterative Binary Search: Book code found at position 3
Recursive Binary Search: Book code found at position 3
```

Problem Faced:

PROBLEM 1: How to continue with the helper after the security guard left.

PROBLEM 2: I was very confused how to implement Recursive Binary search.

Conclusion:

This practical helped me understand the difference between **linear search and binary search** and how both can be implemented using **iterative and recursive approaches**. I learned that linear search checks elements sequentially, while binary search is much faster but requires the data to be sorted. The practical also helped me understand how recursion can repeatedly reduce the search range until the target is found or the range becomes empty.

Key Questions / Analysis / Interpretation to be Evaluated During/After Implementation

1. **[Problem 1]** If the target plate appears near the end of a very long list, your solution still checks every plate before it. Can you think of any property the input would need to have for you to skip checking some plates entirely? What does Problem 2.2 tell you about this?

Ans. If the list has some useful ordering property, we could skip checking some plates instead of checking every plate. For example, if the plates were sorted, we could use binary search to eliminate half of the list at each step. This connects to Problem 2.2, which states that binary search requires the input list to be sorted.

2. **[Problem 2]** What is the consequence of applying your solution to a list that is not sorted? Would it always give a wrong answer, or only sometimes? Explain your reasoning without running any example.

Ans. If the list is not sorted, binary search is not guaranteed to work correctly. It may sometimes find the target by chance, but it can also eliminate the half containing the target and return an incorrect result. Therefore, the list must be sorted before applying binary search.